Assignment 4: Machine Translation with LSTM, GRU and Encoder–Decoder Models

CS4101 | CS6103 — English → Hindi on the IIT Bombay corpus

Anshul Kaushal (2611AI10) Aditya Sharma (2611CS11)
Roland Zohmingthana (2611MC03)

November 2026

Abstract

We implement vanilla RNN, LSTM and GRU cells from basic tensor operations, verify them against PyTorch to $\leq 4 \times 10^{-15}$ (values, final states and gradients, on padded batches), and build a configurable encoder–decoder translator (no attention) trained with teacher forcing and decoded greedily. On a fixed-seed subset of 20,000 IIT Bombay English–Hindi pairs of 1–10 tokens, four configurations (LSTM/GRU $\times$ $H \in \{64, 128\}$) are trained for 10 epochs. GRU-128 is selected by validation BLEU-4 (2.56) and reaches **2.20 BLEU-4** on the test set (loss 3.368, perplexity 29.0, token accuracy 48.5%). Our own corpus BLEU-4 equals sacrebleu exactly on every evaluation. Absolute BLEU is low because the short end of the corpus is dominated by glossary entries and UI strings and 26.6% of Hindi test tokens are outside the training vocabulary; we analyse these limits, the gradient flow of the three cells, exposure bias, SVO→SOV reordering through a fixed-size state, and the stability of the model ranking across seeds.

1 Dataset and tokenisation

Source and cleaning. The official *train* split of `cfilt/iitb-english-hindi` (1,659,083 pairs, only 1,093,624 distinct English sentences) is downloaded as a parquet file with `urllib` (no Hugging Face libraries). The official dev/test splits are news sentences that are nearly all longer than 10 tokens, so all three of our splits are drawn from the train split. A pair is kept if the English side contains only ASCII letters, digits and common punctuation (this removes software strings with `%s`, `_`, `&`, markup and mis-encoded text), the Hindi side only Devanagari, digits and punctuation with at least one Devanagari letter (removes untranslated English and mojibake, i.e. UTF-8 Hindi mis-decoded as Latin-1), both sides have 1–10 tokens, and neither side is more than $3 \times +2$ longer than the other (Table 1).

Fixed-seed subset. The 667,057 clean pairs are permuted with `numpy.random.default_rng(42)`; walking through the permutation we keep only the first occurrence of each English sentence (one randomly chosen translation per duplicated source; this also guarantees that no source sentence occurs in two splits) and stop at 20,000 pairs, split 16,000 / 2,000 / 2,000 (train/validation/test).

Filter step	pairs
invalid English characters	126,947
invalid Hindi characters	92,625
outside 1–10 tokens	769,421
length ratio	3,033
kept (319,291 distinct English)	**667,057**

Table 1: Cleaning funnel on the 1,659,083 raw training pairs.

Tokenisation. English: NFKC, lower-casing, curly→straight quotes and a regular expression that keeps words (with clitics such as `'s`), numbers and single punctuation marks. Hindi needs a Devanagari-aware rule: vowel signs (ि, ा), *virama* ्, *anusvara* ं and *nukta* ़ are combining marks that Python's `\w` does not match, so a `\w+` tokeniser would split हिन्दी into fragments. We (i) apply Unicode NFC and remove zero-width joiners so each word has one code-point sequence; (ii) handle **sentence-ending punctuation**: the corpus ends Hindi sentences with the *danda* ।, an ASCII "." or a pipe "|" inconsistently; a sentence-final "."/"|" is mapped to । and ।/॥ are always split off as separate tokens. Otherwise है।, है. and है would be three vocabulary items, inflating the

vocabulary and UNK rate, and BLEU would penalise a correct final verb because of an arbitrary punctuation choice; (iii) every other punctuation mark is a separate token and maximal runs of Devanagari letters/signs form words. The same tokeniser produces the training targets, the references and the hypotheses; BLEU is computed on these pre-tokenised strings (`tokenize="none"` in sacrebleu).

Vocabulary and statistics. Vocabularies are built on the training split with `min_freq` = 2 (singletons become `<unk>`, which also teaches the model to handle unseen words) plus four specials: **English 4,899** and **Hindi 4,974** entries. Table 2 gives the split sizes and unknown-token rates. Mean length is only ≈ 3.7 tokens: 47% of test sources have 1–2 tokens, because the short end of IITB consists largely of dictionary/glossary entries (*pistachio nut* → पिस्ता) and UI strings (*failed to create user*). Hindi has more types and a higher UNK rate than English: it inflects nouns, adjectives and verbs for gender, number and case (गया/गई/गए) and spells loanwords in several ways (फ़ाइल/फाइल, पहुंच/पहुँच). 70% of Hindi training types (11,357 of 16,327) occur only once.

split	language	sentences	tokens	types	`<unk>` tokens	UNK rate
train	English	16,000	57,478	14,303	9,408	16.37%
train	Hindi	16,000	58,556	16,327	11,357	19.40%
val	English	2,000	7,367	3,331	1,757	23.85%
val	Hindi	2,000	7,593	3,604	1,975	26.01%
test	English	2,000	7,521	3,374	1,735	23.07%
test	Hindi	2,000	7,690	3,704	2,042	26.55%

Table 2: Split sizes and token-level unknown rates (vocabulary from train, `min_freq`=2; the train rate is the share of singleton tokens).

2 Task 1: recurrent cells and encoder–decoder

2.1 Cells, initialisation and padding

All cells use only matrix products, additions, element-wise products, `sigmoid` and `tanh`, with PyTorch's weight layout ($W_{ih} \in \mathbb{R}^{GH \times d}$, $W_{hh} \in \mathbb{R}^{GH \times H}$, two biases; gate order i, f, g, o for the LSTM and r, z, n for the GRU):

$$\text{RNN: } h_t = \tanh(W_{ih}x_t + b_{ih} + W_{hh}h_{t-1} + b_{hh})$$

$$\text{LSTM: } [i, f, g, o] = W_{ih}x_t + b_{ih} + W_{hh}h_{t-1} + b_{hh}, \quad c_t = \sigma(f) \odot c_{t-1} + \sigma(i) \odot \tanh(g), \quad h_t = \sigma(o) \odot \tanh(c_t)$$

$$\text{GRU: } r = \sigma(W_{ir}x_t + b_{ir} + W_{hr}h_{t-1} + b_{hr}),\ z = \sigma(W_{iz}x_t + b_{iz} + W_{hz}h_{t-1} + b_{hz}),$$
$$n = \tanh\big(W_{in}x_t + b_{in} + r \odot (W_{hn}h_{t-1} + b_{hn})\big), \quad h_t = (1 - z) \odot n + z \odot h_{t-1}.$$

Initialisation: every weight and bias is drawn i.i.d. from $\mathcal{U}(-1/\sqrt{H}, 1/\sqrt{H})$ using the global torch generator, which is seeded immediately before each model is built. This keeps gate pre-activations at unit order so sigmoids/tanh start unsaturated. We deliberately do not add a forget-gate bias of +1, so that all three cells start from the same distribution in the gradient experiment. Embeddings use PyTorch's $\mathcal{N}(0, 1)$ default (padding row zero) and the output layer its default uniform initialisation.

Padded batches: a `RecurrentLayer` runs a cell over time given the true lengths L_b. For $t \geq L_b$ the state of sequence b is frozen with `torch.where` and its output is zero, so the returned final state is the state after the last real token (the semantics of `pack_padded_sequence`) and padding affects neither the state nor the gradients.

2.2 Verification against PyTorch

We copy our weights into `nn.{RNN,LSTM,GRU}Cell` and `nn.{RNN,LSTM,GRU}`, and compare in float64 on a padded batch with lengths $(7, 5, 3, 1, 6)$ fed to the built-in layer through `pack_padded_sequence`. This is the only use of built-in recurrent layers. Table 3: all differences are at machine precision and padded outputs are exactly zero.

cell	single step	outputs	final state(s)	padded out = 0	∇ input	∇ weights
RNN	1.1e-16	1.1e-16	1.1e-16	yes	2.2e-16	3.6e-15
LSTM	2.2e-16	5.6e-17	1.1e-16	yes	1.1e-16	5.6e-16
GRU	1.1e-16	1.9e-16	1.7e-16	yes	2.2e-16	1.8e-15

Table 3: Maximum absolute difference between our cells and PyTorch (float64).

2.3 Parameter count for $d = 128$, $H = 256$

A cell with G gate blocks has $G(dH + H^2 + 2H)$ parameters: **RNN 98,816**, **GRU 296,448** ($G = 3$), **LSTM 395,264** ($G = 4$); the counted `numel` agrees. The ratio LSTM/GRU is exactly 4/3 for every d, H: each gate block (input matrix, recurrent matrix, two biases) has the same size in both cells, and the LSTM has four blocks (input, forget, output gate, cell candidate) while the GRU has three (reset, update, candidate). The GRU couples "forget" and "input" into one update gate (z and $1 - z$) and has no separate cell state, hence no output gate. The second bias is required in the GRU because b_{hn} sits inside the reset product; for the LSTM it is redundant but kept for PyTorch compatibility.

2.4 Gradient flow

For $T \in \{10, 25, 50, 100\}$, $B = 16$, $d = 32$, $H = 64$ and $x \sim \mathcal{N}(0, I)$ we compute $\|\partial \|h_T\|_2^2 / \partial x_1\|_2$ per sequence, average over the batch and over seeds 0–4. For seed s the cell is initialised after `torch.manual_seed(s)` and the inputs come from a separate generator seeded with s, so all cells see identical inputs. We use float64 on CPU to avoid underflow (Figure 1, Table 4).

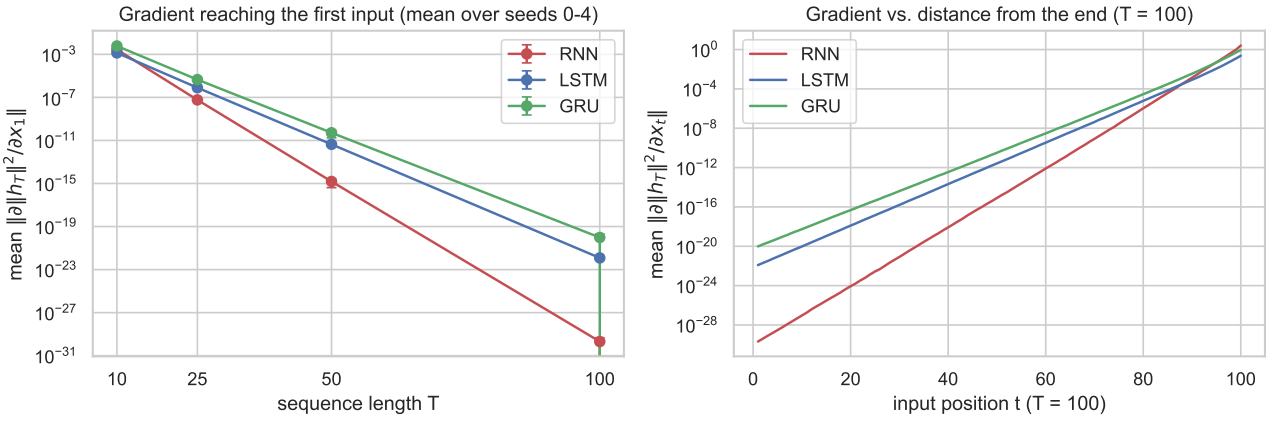


Figure 1: Gradient-flow comparison for vanilla RNN, LSTM and GRU: mean gradient norm reaching the first input versus sequence length (left, log scale), and gradient norm by input position for a 100-step sequence (right).

	T	RNN	LSTM	GRU
	10	2.80×10^{-3}	1.36×10^{-3}	5.98×10^{-3}
	25	5.85×10^{-8}	7.94×10^{-7}	4.50×10^{-6}
	50	1.53×10^{-15}	4.33×10^{-12}	5.17×10^{-11}
	100	2.07×10^{-30}	1.22×10^{-22}	9.65×10^{-21}
avg. per-step factor		0.50	0.61	0.63

Table 4: Mean $\|\partial \|h_T\|^2 / \partial x_1\|$ over seeds 0–4; last row: $e^{-\text{slope}}$ of the $T = 100$ log-profile.

Observation. All curves are straight lines on the log scale: the gradient reaching x_1 *vanishes exponentially* in T, since it is a product of $T - 1$ Jacobians $\partial h_t / \partial h_{t-1}$. For the **RNN** each Jacobian is $\text{diag}(1 - h_t^2) W_{hh}$; with $\mathcal{U}(\pm 1/\sqrt{H})$ the spectral radius of W_{hh} is $\approx 1/\sqrt{3} \approx 0.58$ and $\tanh' \leq 1$, so each step halves the gradient (factor 0.50), leaving 10^{-30} after 100 steps. The **LSTM** starts *below* the RNN at $T = 10$, because the output gate and the extra $\tanh(c_t)$ each scale h_T by about 0.5, but it decays more slowly and is eight orders of magnitude larger at $T = 100$. Its cell state has an additive path $c_t = f_t \odot c_{t-1} + \ldots$ whose Jacobian is just $\text{diag}(f_t)$, with no weight matrix and no squashing. The **GRU** is largest at every T: $h_t = z \odot h_{t-1} + (1 - z) \odot n$ is a gated identity path, and there is no output gate. At initialisation all gates are $\approx \sigma(0) = 0.5$, so gated cells *also* vanish; gating is no guarantee. The difference is that their per-step factor is a learnable, input-dependent gate that training can drive towards 1 (a forget bias of +1 alone gives $\sigma(1) = 0.73$), whereas the RNN's factor is fixed by W_{hh} and tanh saturation.

2.5 Encoder–decoder model

The **encoder** is an embedding followed by `num_layers` of our recurrent layers (LSTM or GRU), run over the true source lengths (source + `<eos>`). Its final state (h, and c for the LSTM) is the only information passed

on: a fixed-size summary with no attention. The **decoder** is an embedding followed by the same cell type initialised with that state, then dropout and a linear layer to the Hindi vocabulary. Configurable options: cell type, embedding size, hidden size, number of layers, dropout. During **training (teacher forcing)** the decoder input at step t is the *reference* token y_{t-1} (`<bos>` at $t = 1$), and the loss is the token cross-entropy over all non-pad targets including `<eos>`. For **inference (greedy)** we start from `<bos>`, feed back the arg-max token, and stop when all sentences have emitted `<eos>` or after 20 steps.

2.6 Teacher forcing versus free-running decoding

Under teacher forcing the decoder is always conditioned on the true prefix $y_{<t}$; in free running it is conditioned on its own predictions $\hat{y}_{<t}$. During training it never sees its own mistakes (*exposure bias*), so an early error moves it into a state distribution it was never trained on, and the errors compound. **Example** (final model, validation): *"failed to create user"*, reference उपयोक्ता बनाने में विफल. With teacher forcing the arg-max predictions are उपयोक्ता चुनें में विफल `<eos>`: the second word is wrong (चुनें "select" instead of बनाने "create"), but because step 3 is fed the reference बनाने, the model still predicts the correct में विफल. In free running चुनें is fed back. It is the imperative that ends thousands of UI strings, so the decoder emits `<eos>` at once and outputs उपयोक्ता चुनें ("select user"): fluent, but "failed" is lost. Figure 2 quantifies this. Both regimes agree at position 1 (same input), but from position 2 free-running accuracy drops well below teacher-forced accuracy (e.g. 0.20 vs 0.36 at position 5). Teacher-forced accuracy rises again at late positions because long references end in the predictable है । `<eos>`.

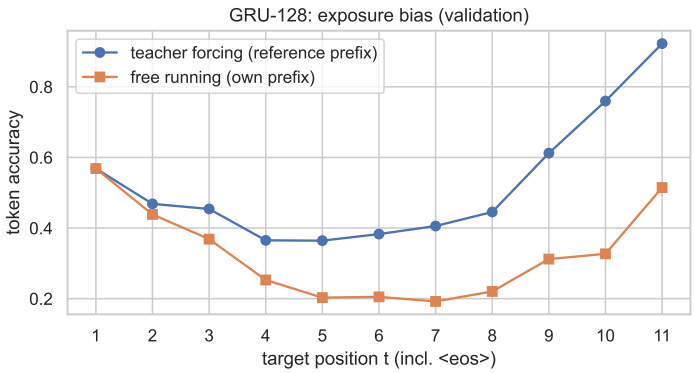


Figure 2: Teacher-forced versus free-running decoding: token accuracy by target position on the validation set for GRU-128.

2.7 Training, metrics and BLEU

Per epoch we log the training loss, perplexity (e^{loss}) and token accuracy (arg-max = reference over non-pad positions, teacher-forced, accumulated with dropout active); the same three on validation; and validation BLEU-4 of greedy translations against the original tokenised references. Reference OOV words are not replaced by `<unk>`, so an output `<unk>` never matches.

Own BLEU-4. Corpus-level BLEU $= \text{BP} \cdot \exp(\frac{1}{4}\sum_{n=1}^{4} \log p_n)$ with clipped n-gram counts summed over the whole corpus before dividing, $p_n = \sum_s \sum_g \min(c_h(g), c_r(g)) / \sum_s \max(|h_s| - n + 1, 0)$, and $\text{BP} = \min(1, e^{1-r/c})$ for total reference length r and hypothesis length c. Optional NIST "exp" smoothing (sacrebleu's default) replaces the k-th zero precision by $1/(2^k \,\text{total}_n)$. **Comparison with sacrebleu** on the same pre-tokenised strings (`tokenize="none"`): the maximum difference over 400 random test corpora (both smoothing modes) is 1.4×10^{-14}, and the two scores agree to within 9×10^{-16} in all 40 per-epoch validation evaluations and on the test set. sacrebleu is used for all model selection. With sacrebleu's default `13a` tokeniser instead, test BLEU would be 1.07 rather than 2.20, because `13a` re-tokenises our strings (e.g. splitting `<unk>` into three tokens). This is why both scores must be computed on the same pre-tokenised text.

Why BLEU under-rewards valid Hindi. Hindi marks grammatical roles with postpositions (ने, को, से, में) rather than position, so constituent order is relatively free. BLEU counts surface n-gram overlap with a single reference, so a meaning-preserving reordering loses most 3- and 4-grams. In our constructed example (reference राम ने सीता को किताब दी ।) the reordering सीता को राम ने किताब दी । (same meaning) scores **35.9**, the synonym पुस्तक for किताब scores 48.9, and the sentence with the *opposite* meaning सीता ने राम को किताब दी । (Sita gave Ram) scores **47.3**, higher than the correct reordering. Spelling variants (गई/गयी, फ़ाइल/फाइल), optional honorific plurals

and the choice between Sanskritised and English loanwords (संगणक/कंप्यूटर) are penalised in the same way, and inflection multiplies the forms that must match exactly.

3 Task 2: cell type and hidden size

Fixed hyperparameters (all four runs). Embedding size 128, one layer, dropout 0.3 (on embeddings and before the output layer), Adam with lr 3×10^{-3} and $\beta = (0.9, 0.999)$, batch size 64, gradient-norm clipping 1.0, 10 epochs, greedy decoding of at most 20 steps, `min_freq` 2, seed 42 for initialisation and shuffling (so all runs see the same batch order). The learning rate and batch size were chosen in a short pilot on the validation set (LSTM-128 with lr 10^{-3} was still improving at epoch 10; lr 3×10^{-3} converged within the budget; batch 32 was only marginally better at twice the cost). The checkpoint of each run is its earliest epoch with maximum validation sacreBLEU.

model	params	time (s)	ckpt ep.	val BLEU	val loss	val ppl	val acc	train loss	train acc	min val-loss ep.
LSTM-64	1,686,382	124.1	9	1.51	3.718	41.18	0.443	3.719	0.414	10
GRU-64	1,661,550	130.1	8	1.50	3.603	36.72	0.452	3.621	0.420	10
LSTM-128	2,169,582	129.7	8	1.65	3.621	37.39	0.461	3.325	0.440	10
GRU-128	2,103,534	129.8	10	**2.56**	**3.446**	**31.37**	**0.473**	2.634	0.491	8

Table 5: Task 2 runs. Time = total training time over 10 epochs (excluding validation decoding); metrics at the checkpoint epoch. Our BLEU equals sacrebleu in every row.

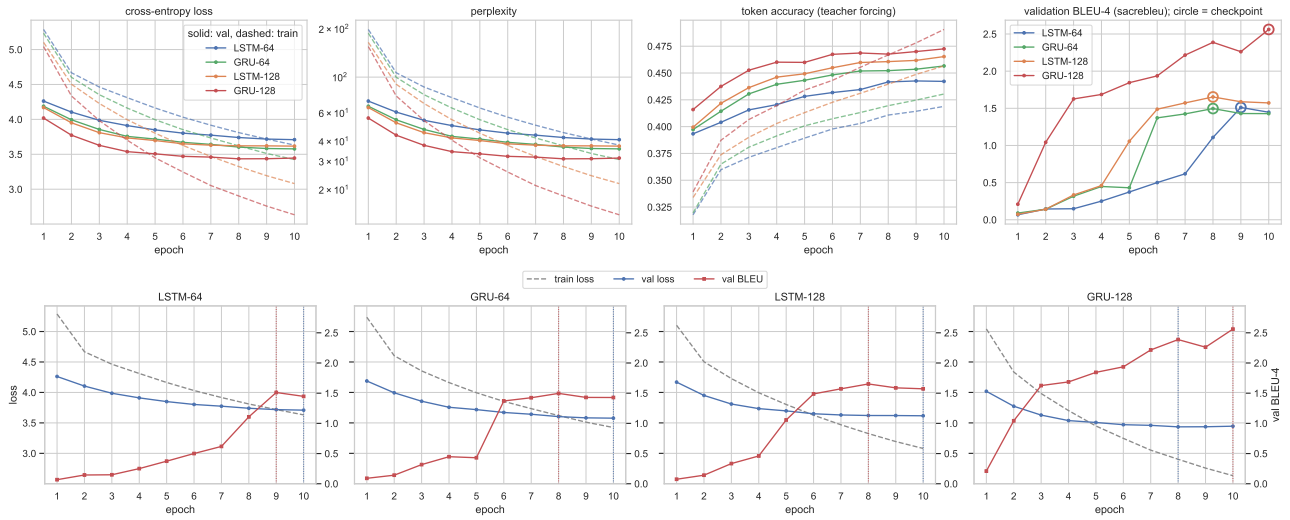


Figure 3: Training and validation metrics for all four configurations (top), and validation loss versus BLEU-4 (bottom). Markers identify the selected checkpoint; dotted lines indicate the minimum validation loss and maximum BLEU-4.

Effect of cell type and hidden size. *Quality:* GRU-128 is clearly best (BLEU 2.56, perplexity 31.4). The other three lie within 0.15 BLEU of each other. A larger H helps both cells (lower validation loss), and the GRU gains more. With ≤ 11 time steps there are no long-range dependencies for the LSTM's separate memory to exploit, and the GRU, with fewer parameters per unit and the strongest gradient path (Table 4), learns faster in 10 epochs: its validation BLEU jumps already at epoch 2–3. *Cost:* the recurrent cells are a small part of each model. For GRU-128, the two embedding tables (627,072 + 636,672) and the output layer (641,646) hold 1.91 M of 2.10 M parameters, and the encoder and decoder GRUs only 99,072 each. This is a direct consequence of the 4.9k/5.0k vocabularies: doubling H adds only 0.44–0.48 M parameters, and LSTM vs GRU differs by $\leq$ 66k. Training time is 130–138 s for every run because, with $T \leq 11$ and small matrices, the GPU is limited by the Python time loop and kernel launches, not by FLOPs (wall-clock times vary by several seconds between executions, so the small differences between runs are not meaningful). *Under/over-fitting:* with dropout active, LSTM-64 still has training loss $\approx$ validation loss (3.63 vs 3.71 at epoch 10), and both are still decreasing, so it is capacity- and epoch-limited (**under-fitting**). GRU-128 reaches training loss 2.63 against validation 3.45, and its validation loss bottoms out at epoch 8 and then rises (the **onset of over-fitting**). The vocabulary statistics explain why this happens so early: 70% of Hindi training types are singletons and 26% of held-out Hindi tokens are unknown, so extra capacity is spent memorising rare training words that never recur, and a quarter of all

reference tokens cannot be produced by any model with this vocabulary (<unk> never matches). The models therefore learn frequent function words, postpositions and sentence frames well, but not content words.

Validation BLEU or validation loss as checkpoint criterion? Across epochs the two are strongly anti-correlated (Spearman -0.94 to -0.99), yet they pick different epochs: for GRU-128 the loss picks epoch 8 and BLEU picks epoch 10 (2.39→2.56 although the loss rises slightly), and for LSTM-64/128 BLEU peaks at epoch 9/8 while the loss is still falling. We consider validation BLEU the better criterion for this task, for three reasons. (i) It evaluates free-running greedy output, which is what is deployed, whereas the loss is teacher-forced and blind to exposure bias. (ii) Loss and token accuracy reward predicting <unk> for the 26% OOV reference tokens, which BLEU correctly never rewards; a model that becomes more confident about <unk> improves its loss without translating better. (iii) It matches the selection and reporting metric. Its drawback, visible in Figure 3, is *noise*. At BLEU ≈ 2 the epoch-to-epoch wiggles (GRU-128: 2.39, 2.26, 2.56; LSTM-64: 1.51, 1.45) are as large as genuine differences, so the smoother validation loss is the more reliable *over-fitting* signal. A combined rule, maximising BLEU among epochs whose loss is within a tolerance of its minimum, would be a sensible compromise.

4 Task 3: final model

Selection and justification. Ranking by checkpoint validation sacreBLEU gives GRU-128 (2.56) > LSTM-128 (1.65) > LSTM-64 (1.51) > GRU-64 (1.50); no tie-break is needed. GRU-128 also has the lowest validation loss and perplexity and the highest token accuracy, so all validation metrics agree. It has fewer parameters than LSTM-128, trains no slower than the others, and is ranked first under every seed tested (below). Its checkpoint is epoch 10.

Test set (evaluated once). BLEU-4 2.20 (own) = **2.20** (sacrebleu, `tokenize="none"`), loss 3.368, perplexity 29.02, token accuracy 48.45%. The test BLEU is a little below the validation BLEU used for selection (2.56), as expected after choosing the maximum of 40 noisy validation measurements.

#	English source	Hindi reference	GRU-128 output
1	web development	वेब विकास	वेब विकास
2	a tuberculosis of a spine	रीढ़ का तपेदिक ।	<unk> का <unk>
3	grand turk	क़िबलाआलम	<unk> <unk>
4	steganalysis software	स्टेगएनालिसिस सॉफ़्टवेयर	<unk> <unk>
5	dagger	डैगर	<unk>
6	plotting head is related to graphics .	आलेखन शीर्ष , ग्राफिक्स से संबंधित होता है ।	<unk> <unk> के लिए <unk> <unk> ।
7	insert advanced send options	उन्नत प्रेषण विकल्प डालें	नया डाक को <unk> करें
8	and i 'll show you in a problem .	मैं आपको सवाल में दिखता हूँ	और मैंने तुमको उससे <unk> <unk> है ?
9	adhesive tape	आसंजक टेप	<unk>
10	acta sociologica	एक्टा सोशियोलॉजिका	<unk> <unk>

Table 6: The first 10 test translations.

Error analysis. Table 7 classifies erroneous test translations by error type. Unknown words dominate: most outputs contain <unk>, and transliterated names and numbers cannot be copied without an attention or copy mechanism.

SVO vs SOV and the fixed-size encoder state. English is SVO; Hindi is SOV with postpositions (ग्रामीण क्षेत्र में, literally "rural area in"). The verb, read second by the encoder, is needed *last* by the decoder, after the subject, the object and their postpositions have been generated. Since the decoder has only the encoder's final H-dimensional vector and no way to look back, the verb's lemma, tense and number must survive in that vector through the whole reordering. Our translations show what survives and what does not.

- **The verb-final template survives.** For test sentences (≥ 4 source tokens) whose reference ends in a verb or auxiliary ($n = 283$), 56.5% of hypotheses also end in one, and only 1.4% contain an auxiliary in a non-final position. Outputs such as यह ... है । or ... जारी की गई । (id 17) have correct SOV shape.
- **The specific content that must cross the reordering is lost.** The verb lemma is replaced or dropped (बनाने→चुनें, missing हटाने), tense and number fall back to the generic है/हैं (ids 12, 40), and the object disappears when the verb is emitted in English position (id 113).

error type	source	output (reference)	diagnosis
unknown word (id 17)	this release issued at 1810 hrs .	यह विज्ञप्ति <unk> बजे जारी की गई । (... 1810 बजे ...)	Perfect except the number: 1810 is OOV and cannot be copied. Same for id 71 (राष्ट्रपति भवन : <unk> for the date 16.08.2017).
word order (id 113)	select to previous line	चुनें के लिए रंग (पिछली पंक्ति चुनें)	The verb "select" is produced *first*, in English VO order, while Hindi needs it last; the object is lost and रंग ("colour") is hallucinated.
number agreement, postposition (id 12)	many saving banks have opened in rural areas .	कई <unk> में <unk> <unk> में <unk> है । (... बैंक खुले हैं ।)	The plural subject (कई ... बैंक) needs plural हैं; the output has singular है, and the locative में is duplicated.
tense/number (id 40)	bengal was its pioneer .	<unk> <unk> <unk> <unk> हैं । (बंगाल इसमें अगुआ था ।)	Past singular था is required; the model emits the most frequent auxiliary, present plural हैं.
repetition (id 138)	union bank of india	बैंक बैंक बैंक की अवधि (यूनियन बैंक ऑफ इंडिया)	The fixed-size state does not track which source words are already translated, so greedy decoding loops on बैंक.
omission (id 131)	choose file or directory to remove :	फ़ाइल या निर्देशिका के लिए चुनें: (... को हटाने के लिए चुनें:)	Verb-final and nearly right, but the purpose verb हटाने ("to remove") and the object marker को are dropped.

Table 7: Erroneous test translations by error type (ids index the test split).

- **Longer sources are worse**. BLEU falls from 5.90 for 1–2-token sources ($n = 938$) to 2.19 for 3–5 tokens ($n = 507$) and 2.28 for 6–10 tokens ($n = 555$). Hypotheses are systematically short (length ratio 0.78–0.85): the more the 128 numbers must hold, the earlier the decoder runs out of information and stops.

An attention mechanism would let the decoder consult the verb's source position exactly when it is generated. With a fixed-size state the model learns the target-side frame but loses source-specific details.

Would the ranking hold under a different seed? We retrained all four configurations with seeds 1 and 2, which changes initialisation and shuffling while keeping the data subset fixed (Table 8). **The winner is robust**: GRU-128 is first under every seed (mean 2.44 ± 0.11), about seven standard deviations above the runner-up mean. **The order of the other three is not**: they lie within a few tenths of a BLEU point, swap places between seeds, and LSTM-64 has one poor seed (0.58). At BLEU ≈ 1.5, on 2,000 sentences of which almost half are too short to contribute 4-grams, such differences are within run-to-run noise. We would therefore trust the claim "GRU-128 is best", but not a claimed ordering among LSTM-64, GRU-64 and LSTM-128 without more seeds or a larger validation set. A different *data* seed (another 20k subset) adds further variance that this experiment does not measure.

model	seed 42	seed 1	seed 2	mean ± std	ranks (42, 1, 2)
LSTM-64	1.51	0.58	1.56	1.22 ± 0.55	3, 4, 4
GRU-64	1.50	1.51	1.90	1.64 ± 0.23	4, 3, 2
LSTM-128	1.65	1.76	1.60	1.67 ± 0.08	2, 2, 3
GRU-128	**2.56**	**2.43**	**2.33**	$\mathbf{2.44 \pm 0.11}$	1, 1, 1

Table 8: Checkpoint validation sacreBLEU under three training seeds.

5 Reproducibility

Seeds are set explicitly for every random choice: subset selection (`default_rng(42)`), parameter initialisation (`torch.manual_seed` before each model), shuffling (`default_rng(seed)` per run), the gradient-flow experiment (seeds 0–4, separate input generator), and the random BLEU test corpora. Training used an NVIDIA RTX 3050 Laptop GPU (PyTorch 2.13.0+cu126, sacrebleu 2.6.0) with `torch.use_deterministic_algorithms(True)`, `cudnn.deterministic=True`, `benchmark=False` and `CUBLAS_WORKSPACE_CONFIG=:4096:8`, so any non-

deterministic kernel would raise an error. Two independent executions produced identical losses, accuracies and BLEU scores (only wall-clock times differ). Results on a CPU or a different GPU/library version can differ in the last bits of floating-point arithmetic, and hence slightly in the trained models. The notebook runs end to end (about 45 minutes, including the seed study) and writes every figure and number in this report to `figures/` and `results/results.json`.

6 Limitations

- Absolute BLEU is low (2.2). This reflects the 1–10-token slice of IITB (glossary entries, UI strings, loose literary translations), 26% Hindi OOV with a word-level vocabulary, 16k training pairs and the assignment's restrictions (no attention, at most 10 epochs, greedy decoding). Subword units (BPE), a copy or attention mechanism, or beam search would address the main error types above but were outside the scope or not allowed.
- BLEU differences below about 0.5 points are within seed noise (Table 8); only the GRU-128 result is robust.
- Training metrics are accumulated with dropout active and are therefore pessimistic. The pilot that fixed lr and batch size used only the validation set and two configurations.
- All splits come from the IITB *train* split after our filtering, so results are not comparable to published IITB test-set numbers.

7 Contributions

member	contribution
Anshul Kaushal (2611AI10)	Data pipeline (download, cleaning, Hindi tokenisation, subset and statistics), encoder–decoder model, training loop and Task 2 experiments.
Aditya Sharma (2611CS11)	RNN/LSTM/GRU cells, verification against PyTorch, parameter counts, gradient-flow experiment, BLEU-4 implementation and sacrebleu comparison.
Roland Zohmingthana (2611MC03)	Final model evaluation, teacher-forcing analysis, error and word-order analysis, seed study, and the report.